

TIP101 | Intro to Technical Interview Prep

Intro to Technical Interview Prep Summer 2026 (@ Section 1 | Tuesdays and Thursdays 3PM - 5PM PT)
Personal Member ID#: 151287

Need help? Post on our [class slack channel](#) or email us at support@codepath.org

[Getting Started](#)[Learning with AI](#)[▶ Course Info](#)[Course Progress](#)[Unit 1](#)[Unit 2](#)[Unit 3](#)[Unit 4](#)[Unit 5](#)[Unit 6](#)[Unit 7](#)[Unit 8](#)[Overview](#)[Session #1](#)[Session #2](#)[Assignment](#)[Cheatsheet](#)[Resources](#)[Report](#)

Unit 4: Session 1

Two Pointer Technique

In this session we will continue to work with strings and lists. Students will develop optimal ways to iterate through these data structures in order to do things like, reverse strings, find the middle of a list, etc.

You can find all resources from today including the session deck, session recording, and more on the [resources tab](#)



Part 1 : Instructor Led Session

We'll spend the first portion of the synchronous class time in large groups, where the instructor will lead class instruction for 30-45 minutes.



Part 2: Breakout Session

In breakout sessions, we will explore and collaboratively solve problem sets in small groups. Here, the **collaboration, conversation, and approach** are just as important as “solving the problem” - please engage warmly, clearly, and plentifully in the process!

In breakout rooms you will:

- Screen-share the problem/s, and verbally review them together
- Screen-share an interactive coding environment, and talk through the steps of a solution approach
 - ProTip: - An Integrated Development Environment (IDE) is a fancy name for a tool you could use for shared writing of code - like Replit.com, Collabed.it, CodePen.io, or other - your staff team will specify which tool to use for this class!
- Screen-share an implementation of your proposed solution
- Independently follow-along, or create an implementation, in your own IDE.

Your program leader/s will indicate which code sharing tool/s to use as a group, and will help break down or provide specific scaffolding with the main concepts above.

► **Note on Expectations**

Problem Solving Approach

To build a long-term organized approach to problem solving, we'll start with three main steps. We'll refer to them as **UPI: Understand, Plan, and Implement**.

We'll apply these three steps to most of the problems we'll see in the first half of the course.

We will learn to:

- **Understand** the problem,
- **Plan** a solution step-by-step, and
- **Implement** the solution

► **Comment on UPI**

► **UPI Example**

To jumpstart your journey into the two-pointer technique, we've put together a guide to common variations of the technique you will use throughout Unit 4 breakout problems. Use this cheatsheet as a quick reference guide as you work through the problems below.

Problem Set Version 1

▼ Problem 1: Prime Number

Write a function `is_prime()` that takes in a positive integer `n` and returns `True` if it is a prime number and `False` otherwise. A **prime number** is a number greater than 1 that has no positive divisors other than 1 and itself.

```
def is_prime(n):  
    pass
```

Example Usage:

```
print(is_prime(5))  
print(is_prime(12))  
print(is_prime(9))
```

Example Output:

```
True  
False  
False
```

▼ ✨ AI Hint: While Loops

Key Skill: Use AI to explain code concepts

This exercise may require you to use a while loop. If you aren't sure what a while loop is, or how it differs from a for loop, try asking ChatGPT or GitHub copilot:

"You're an expert computer science tutor. What is the difference between a for loop and a while loop in Python? Please provide examples of each."

Then open the next hint to see our answer!

▼ 💡 Hint: While Loops (more detail)

For loops and while loops are both used to repeat a block of code, but for loops are iteration based (used to iterate over a sequence) while while loops continues to execute as long as a specified condition is true.

In general, we should use a while loop instead of a for loop when:

- we do not know beforehand the exact number of repetitions we want to do

```
# For Loop: Repeat for however many elements there are in the list
for elt in len(lst):
    print(elt)

# While Loop: Repeat until fruit = 'banana'
# but we're not exactly sure when that will be
while fruit != 'banana':
    print('Guess my favorite fruit!')
    # Input function returns user's guess
    fruit = input('Enter your guess: ')
```

- we want greater control over our loop variable

```
# For Loop: Loop variable increases by two each time
for num in range(1, 10, 2):
    print(num)

# While Loop: Conditionally increase loop variable by different amounts
num = 1
while num < 10:
    print(num)
    if num % 2 == 0:
        num += 1
    else:
        num += 2
```

- we want to mutate a variable during the iteration process

```
# For Loop: Mutating a list inside a for loop can cause unexpected results!
# Can make loop variable out of sync with list contents
for elt in lst:
    if elt == 'b':
        lst.remove('b')

# While Loop: The condition and value of the list is checked
# at the beginning of each iteration, making it safe to modify
while 'b' in lst:
    lst.remove('b')
```

Close Section

▼ Problem 2: Two-Pointer Reverse List

Write a function `reverse_list()` that takes in a list `lst` and returns elements of the list in reverse order. The list should be reversed *in-place* without using list slicing (e.g. `lst[::-1]`).

Instead, use the **two-pointer approach**, which is a common technique in which we initialize two variables (also called a pointer in this context) to track different indices or places in a list or string, then moves the pointers to point at new indices based on certain conditions. In the most common variation of the two-pointer approach, we initialize one variable to point at the beginning of a list and a second variable/pointer to point at the end of list. We then shift the pointers to move inwards through the list towards each other, until our problem is solved or the pointers reach the opposite ends of the list.

```
def reverse_list(lst):
    pass
```

Example Input: `[1, 2, 3, 4, 5]`

Example Output: `[5, 4, 3, 2, 1]`

▼ 💡 Hint: Two Pointer Technique

This problem requires the use of the two pointer technique. For a more in-depth review of the two-pointer technique, check out the [Unit 4 Cheatsheet](#), specifically the opposite direction variation.

Close Section

▼ Problem 3: Evaluating Solutions

The `reverse_list()` problem can also be solved without using the two pointer technique (as you may have seen it in previous units)! Evaluate the time and space complexity of your two-pointer solution.

Then, evaluate the time and space of the following solution:

```
def reverse_list(lst):  
    # Create a new reversed list  
    reversed_lst = lst[::-1]  
    # Copy the elements back into the original list  
    for i in range(len(lst)):  
        lst[i] = reversed_lst[i]
```

Which has better time complexity?

Which has better space complexity?

▼ 💡 Hint: Big O (Time & Space Complexity)

Big O notation is a mathematical notation in computer science used to describe the the time and space complexity of an algorithm. Time complexity is the amount of time an algorithm or function takes to run in comparison to the size of the input data. Space complexity is the amount of extra memory or space an algorithm or function needs to complete its task in comparison to the size of the input data.

For your convenience, we've included a summary of the three most common Big O functions below.

Common Big O includes:

- **$O(1)$ - Constant Time** No matter the size of your input data, the function takes a fixed amount of time or memory to complete its task.

Example: Summing two numbers

```
def sum(a, b):  
    return a + b
```

It takes the computer roughly the same amount of time to sum `a` and `b` no matter how large the two numbers are.

- **$O(n)$ - Linear Time** The amount of time or memory your function needs grows linearly with the size of your input data.

Example: Printing each item in a list

```
def print_list(lst):  
    for item in lst:  
        print(item)
```

The computer has to perform one extra print statement for each extra item there is in the list, so the length of time it takes to print the list will be proportional to the number of items in the list. We expect that it will take 1000 times longer to print a list with 1000 elements than it will to print a list with just 1 element.

- **$O(n^2)$ - Quadratic Time** The amount of time or memory your function needs grows quadratically with the size of your input data.

Example: Finding Duplicates Using a Nested For Loop

```
def find_duplicates(lst):  
    n = len(lst)  
    for i in range(n):  
        for j in range(i + 1, n):  
            if lst[i] == lst[j]:  
                print(f"Duplicate found: {lst[i]}")  
                return True  
    return False
```

The function compares each element in the list to every other element in the list, which means we perform roughly n^2 comparisons where n is the length of our input list `lst`, so it will take n^2 time to

complete all comparisons. We can expect that for a list of size 2, we will perform roughly 4 comparisons whereas for a list of size 10 we will perform roughly 100 comparisons.

▼ ✨ AI Hint: Decoding Big O

Key Skill: Use AI to explain code concepts

Big O is a big topic, and kind of tricky to wrap your head around! If you're feeling confused, try asking an AI tool like ChatGPT or GitHub Copilot to explain it to you:

"You're an expert computer science tutor for a Python-based technical interviewing prep course. Can you use an analogy to help me understand Big O notation? Please explain the concept of time and space complexity in a way that is easy to understand."

Once it gives you an answer, you can ask follow-up questions to clarify any points that are still confusing. Be patient with yourself, and remember that this is a complex topic that takes time to fully understand!

Close Section

▼ Problem 4: Move Even Integers

Write a function `sort_list_by_parity()` that takes in an integer list `nums` as a parameter and moves all the even integers at the beginning of the list followed by all the odd integers. The function returns any list that satisfies this condition.

```
def sort_array_by_parity(nums):  
    pass
```

Example Usage:


```
nums = [3,1,2,4]
nums2 = [0]
print(sort_array_by_parity(nums))
print(sort_array_by_parity(nums2))
```

Example Output:

```
[2,4,3,1]
# Additional acceptable outputs are [4,2,3,1], [2,4,1,3], and [4,2,1,3]
[0]
```

Close Section

▼ Problem 5: Palindrome

Write a function `first_palindrome()` that takes in a list of strings `words` as a parameter and returns the first palindromic string in the list. A string is **palindromic** if it reads the same forward and backward. If there is no such string, return an empty string `""`

```
def first_palindrome(words):
    pass
```

Example Usage:

```
words = ["abc","car","ada","racecar","cool"]
palindrome1 = first_palindrome(words)
print(palindrome1)

words2 = ["abc","racecar","cool"]
palindrome2 = first_palindrome(words2)
print(palindrome2)

words3 = ["abc", "def", "ghi"]
palindrome3 = first_palindrome(words3)
print(palindrome3)
```

Example Output:

ada
racecar

▼ 💡 Hint: Helper Functions

This problem may benefit from a helper function!

If you find your functions getting too long or performing lots of different tasks, it might be a good indicator that you should add a helper function. Helper functions are functions we write to implement a subtask of our primary task.

We can then call our function inside of our main function, to keep our main function shorter and easy to read.

Example: Calculating a Bill Total

```
# Helper Function: Calculate the price of one item
def calculate_price(item_price, quantity):
    return item_price * quantity

# Primary Function: Calculate total cost of bill
def calculate_total(bill):
    total = 0
    for item_price, quantity in bill.items():
        # Call helper function
        total += calculate_price(item_price, quantity)
    return total
```

Creating helper functions helps us follow the Single Responsibility Principle (SRP) which says that a function should perform only a single task or action. SRP helps us create clean, readable, and maintainable code!

▼ Problem 6: Remove Duplicates with $O(1)$

Write a function `remove_duplicates()` that takes in a *sorted* list of integers `nums` as a parameter and removes the duplicates in-place such that each element appears only once. Do not allocate extra space for another array; you must do this by modifying the input list with **$O(1)$** extra memory. The function returns the new length of the list.

```
def remove_duplicates(nums):  
    pass
```

Example Usage:

```
nums = [1,1,2,3,4,4,4,5]  
print(nums)  
print(remove_duplicates(nums))  
print(nums) # same list
```

Example Output:

```
[1,1,2,3,4,4,4,5]  
5  
[1,2,3,4,5]
```

Close Section

Problem Set Version 2

- ▶ Problem 1: Perfect Number
- ▶ Problem 2: 2-Pointer Palindrome
- ▶ Problem 3: Evaluate Palindrome
- ▶ Problem 4: Make Palindromes
- ▶ Problem 5: Reverse Vowels
- ▶ Problem 6: Two-Pointer Remove Element

Problem Set Version 3

- ▶ Problem 1: Highest Exponent
- ▶ Problem 2: Two-Pointer Target Sum
- ▶ Problem 3: Evaluate Two Sum
- ▶ Problem 4: Two-Pointer Reverse Letters
- ▶ Problem 5: Reverse Prefix
- ▶ Problem 6: Squash Spaces